14.3 出站消息处理

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~280,000 tokens，输出 ~20,000 tokens（本章合计）

Agent 生成回复后，消息需要经过出站处理流水线才能发送到用户的通道。这个流程包括消息分块、确认反应、打字指示器和会话标签生成等步骤。

14.3.1 消息分块（Chunking）算法与通道限制

AI Agent 的回复可能非常长——一个详细的代码分析或技术解释可能达到数千甚至上万字符。但每个消息通道都有单条消息的长度限制：

通道	单条消息上限	分块上限
Discord	2000 字符	2000
Telegram	4096 字符	4000
WhatsApp	~65536 字符	4000
Slack	40000 字符	4000
Signal	~6000 字符	4000

回复超过通道限制时，需要将其**分块**（Chunk）成多条消息依次发送。分块不是简单地按字符数截断——它需要考虑 Markdown 格式的完整性。

每个通道的出站适配器定义了自己的分块器（Chunker）和分块模式：

```
// src/channels/plugins/types.adapters.ts（出站适配器类型）
export type ChannelOutboundAdapter = {
  chunker?: ((text: string, limit: number) => string[]) | null;
  chunkerMode?: "text" | "markdown";
  textChunkLimit?: number;
};
```

大多数通道使用 `"text"` 模式的纯文本分块（`chunkText`），而 Telegram 由于其 HTML 格式支持，使用 `"markdown"` 模式的专用分块器（`markdownToTelegramHtmlChunks`）：

```
// src/channels/plugins/outbound/telegram.ts
export const telegramOutbound: ChannelOutboundAdapter = {
  chunker: markdownToTelegramHtmlChunks,
  chunkerMode: "markdown",
  textChunkLimit: 4000,
};

// src/channels/plugins/outbound/whatsapp.ts
export const whatsappOutbound: ChannelOutboundAdapter = {
  chunker: chunkText,
  chunkerMode: "text",
  textChunkLimit: 4000,
};

// src/channels/plugins/outbound/discord.ts
export const discordOutbound: ChannelOutboundAdapter = {
  chunker: null,  // Discord 有自己的内置分块逻辑
  textChunkLimit: 2000,
};
```

Discord 和 Slack 的 `chunker` 设置为 `null`，表示它们使用自己客户端库内置的分块逻辑，而不是 OpenClaw 的通用分块器。

14.3.2 确认反应（ACK Reactions）

确认反应是一种轻量级的用户反馈机制：Gateway 收到消息并开始处理时，会在原始消息上添加一个表情反应（如 "眼睛" 表情），让用户知道消息已被接收。

`src/channels/ack-reactions.ts` 实现了确认反应的门控逻辑：

```ts
// src/channels/ack-reactions.ts
export type AckReactionScope = "all" | "direct" | "group-all" | "group-mentions" | "off

export function shouldAckReaction(params: AckReactionGateParams): boolean {
  const scope = params.scope ?? "group-mentions";  // 默认只在群组被提及时确认
  if (scope === "off" || scope === "none") return false;
  if (scope === "all") return true;
  if (scope === "direct") return params.isDirect;
  if (scope === "group-all") return params.isGroup;
  if (scope === "group-mentions") {
    // 仅在可提及的群组 + 被提及时确认
    if (!params.isMentionableGroup) return false;
    if (!params.requireMention || !params.canDetectMention) return false;
    return params.effectiveWasMentioned || params.shouldBypassMention === true;
  }
  return false;
}
```

默认行为是 `"group-mentions"` ——只在群组中被 @提及时才发送确认反应。这避免了在私聊中产生冗余的反应（私聊不需要确认，因为回复本身就是确认），同时在群组中给用户明确的反馈。

确认反应还支持**回复后移除**——Agent 发出回复后，之前添加的确认反应会被自动移除：

```ts
// src/channels/ack-reactions.ts
export function removeAckReactionAfterReply(params: {
  removeAfterReply: boolean;
  ackReactionPromise: Promise<boolean> | null;
  ackReactionValue: string | null;
  remove: () => Promise<void>;
}) {
  if (!params.removeAfterReply) return;
  if (!params.ackReactionPromise || !params.ackReactionValue) return;
  // 等确认反应实际发送后再移除
  void params.ackReactionPromise.then((didAck) => {
    if (!didAck) return;
    params.remove().catch((err) => params.onError?.(err));
  });
}
```

14.3.3 打字指示器（Typing Indicators）

许多通道支持"对方正在输入"的提示。OpenClaw 在 Agent 处理消息期间持续发送打字指示器，让用户知道 AI 正在思考。

`src/channels/typing.ts` 提供了打字回调的工厂函数：

```
// src/channels/typing.ts
export type TypingCallbacks = {
  onReplyStart: () => Promise<void>;  // 开始回复时调用
  onIdle?: () => void;                // 空闲时调用（停止打字指示器）
};

export function createTypingCallbacks(params: {
  start: () => Promise<void>;         // 启动打字指示器
  stop?: () => Promise<void>;         // 停止打字指示器
  onStartError: (err: unknown) => void;
  onStopError?: (err: unknown) => void;
}): TypingCallbacks {
  const onReplyStart = async () => {
    try {
      await params.start();
    } catch (err) {
      params.onStartError(err);
    }
  };

  const onIdle = params.stop
    ? () => {
        void params.stop().catch((err) =>
          (params.onStopError ?? params.onStartError)(err)
        );
      }
    : undefined;

  return { onReplyStart, onIdle };
}
```

设计上的一个细节： `onIdle` 回调是**可选的**。这是因为某些通道（如 Telegram）的打字指示器会自动在一定时间后过期，不需要显式停止。而其他通道（如 Discord）需要手动停止打字状态。

打字指示器的错误不会中断消息处理流程——它只是一个辅助性的用户体验功能，即使失败也不应影响 Agent 的正常回复。

14.3.4 会话标签与回复前缀

会话标签

`src/channels/conversation-label.ts` 为每个会话生成一个人类可读的标签，用于在 UI 和日志中标识会话：

```
// src/channels/conversation-label.ts
export function resolveConversationLabel(ctx: MsgContext): string | undefined {
  // 优先使用显式设置的标签
  const explicit = ctx.ConversationLabel?.trim();
  if (explicit) return explicit;

  // 线程使用线程标签
  const threadLabel = ctx.ThreadLabel?.trim();
  if (threadLabel) return threadLabel;

  // 私聊使用发送者名称
  const chatType = normalizeChatType(ctx.ChatType);
  if (chatType === "direct") {
    return ctx.SenderName?.trim() || ctx.From?.trim() || undefined;
  }

  // 群组使用频道名/群组主题/空间名
  const base = ctx.GroupChannel?.trim() || ctx.GroupSubject?.trim()
    || ctx.GroupSpace?.trim() || ctx.From?.trim() || "";
  if (!base) return undefined;

  // 纯数字 ID 或 WhatsApp 群组 ID 时追加 id: 后缀
  const id = extractConversationId(ctx.From);
  if (id && shouldAppendId(id) && base !== id && !base.includes(id)) {
    return `${base} id:${id}`;
  }
  return base;
}
```

标签生成遵循"尽可能有意义"的原则：私聊显示对方的名字（如 "Alice"），群组显示群组名称（如 "#general"），如果只有数字 ID 则追加 `id:` 前缀以便识别（如 "工程群 id:12345"）。

回复前缀

`src/channels/reply-prefix.ts` 可以在 Agent 的每条回复前添加前缀，通常用于标注正在使用的模型或身份信息：

```
// src/channels/reply-prefix.ts
export function createReplyPrefixContext(params: {
  cfg: OpenClawConfig;
  agentId: string;
  channel?: string;
  accountId?: string;
}): ReplyPrefixContextBundle {
  const prefixContext: ResponsePrefixContext = {
    identityName: resolveIdentityName(cfg, agentId),
  };

  const onModelSelected = (ctx: ModelSelectionContext) => {
    // 在模型选择后更新前缀上下文
    prefixContext.provider = ctx.provider;
    prefixContext.model = extractShortModelName(ctx.model);
    prefixContext.modelFull = `${ctx.provider}/${ctx.model}`;
    prefixContext.thinkingLevel = ctx.thinkLevel ?? "off";
  };

  return {
    prefixContext,
    responsePrefix: resolveEffectiveMessagesConfig(cfg, agentId, {
      channel: params.channel, accountId: params.accountId,
    }).responsePrefix,
    responsePrefixContextProvider: () => prefixContext,
    onModelSelected,
  };
}
```

回复前缀是**可配置的模板**，可以包含变量如 `{{model}}` 、 `{{provider}}` 、 `{{identityName}}` 等。例如配置 `responsePrefix: "[{{model}}] "` 会在每条回复前添加 `[claude-4-opus]` 这样的模型标识。

本节小结

出站消息处理流水线将 Agent 的原始回复转化为用户在各通道中看到的最终消息：

1. **智能分块**：根据通道限制自动拆分长消息，支持纯文本和 Markdown 两种分块模式
2. **确认反应**：在消息被接收时添加表情反应，回复后自动移除，默认只在群组被提及时启用
3. **打字指示器**：Agent 处理期间持续发送打字状态，错误不影响主流程
4. **会话标签**：为每个会话生成人类可读的标签，方便在 UI 和日志中识别
5. **回复前缀**：可配置的模板系统，在回复前添加模型、身份等标注信息